

Technology Stack & Architecture Decision Records (ADR)

RetailSync: Centralized Super Shop Warehouse Management System

Subtitle: Engineering Stack Selection, Component Standards, and Architectural Decision Records

Course: SE-231 (Software System Analysis & Design / Capstone Project 2)

Academic Institution: Daffodil International University (DIU), Department of Software Engineering

Author: Raisul Islam Likhon (Section: SWE-44D)

Date: September 2026 | Version: 1.0.0-RELEASE

1. Curated Technology Stack Matrix

RetailSync is built using modern, open-source, battle-tested software engineering technologies carefully curated for high throughput, sub-second latency, developer productivity, and near-zero software licensing costs:

+-----+ RETAILSYNC STACK +-----+			
FRONTEND PWA		Next.js 14 / React 18, Tailwind CSS, TanStack Query, Zustand	
BARCODE ENGINES		ZXing / Html5-QRCode + Bluetooth HID Hardware Scanner Profiles	
BACKEND API		Python 3.11+ / FastAPI (Asynchronous ASGI), Pydantic v2	
DATABASE CORE		PostgreSQL 16 (ACID 3NF Normalized, Composite B-Tree Indexes)	
IN-MEMORY CACHE		Redis 7 (Idempotency Keys, Session Store, Distributed Rate Locks)	
DECISION ENGINE		NumPy, SciPy (Greasley Variance, EOQ), Scikit-learn (Isolation)	
INFRA & DEVOPS		Docker, Docker Compose, Nginx Reverse Proxy, GitHub Actions CI/CD	
TESTING FRAMEWORKS		Pytest, HTTPX AsyncClient, Locust (Concurrency Stress Testing)	
+-----+			

1.1 Detailed Technology Specifications

Architecture Tier	Primary Technology	Selected Version	Strategic Justification for Capstone & Production
Frontend Framework	Next.js / React	14.2+ (App Router)	Single Page Application (SPA) speed with server-side rendering (SSR) for dashboards; eliminates page-reload latency on mobile scanning terminals.
Styling & Design System	Tailwind CSS	3.4+	Rapid component styling; ultra-clean modern aesthetic; utility-first layout supporting high-contrast floor themes.

Architecture Tier	Primary Technology	Selected Version	Strategic Justification for Capstone & Production
Client State Management	Zustand	4.5+	Extremely lightweight (1.1 kB), minimalist state store for managing active picker sessions, scan queues, and cart states without Redux boilerplate.
Data Fetching & Cache	TanStack Query	v5.0+	Automatic background refetching, query caching, and optimistic UI updates for real-time warehouse bin occupancy rendering.
Barcode Scanner Integration	Html5-QRCode / ZXing	Latest Stable	Pure JavaScript barcode engine reading 1D (EAN-13, Code 128, GS1) and 2D (QR) barcodes directly from smartphone cameras and Bluetooth trigger guns.
Backend API Framework	Python FastAPI	0.111+ (Python 3.11)	High-performance asynchronous execution on Starlette/uvloop; automatic OpenAPI 3.1 / Swagger documentation; deep integration with scientific math libraries.
Data Validation & Typing	Pydantic	v2.7+ (Rust core)	Blazing-fast request payload parsing and validation with strict type enforcement for financial quantities and barcode payloads.
Database ORM & Driver	SQLAlchemy + asyncpg	2.0+ / 0.29+	Asynchronous database access with connection pooling; raw SQL query performance for complex row-locking operations.
Core Relational Database	PostgreSQL	16.3	World-class ACID transaction guarantees, robust row-level locking (SELECT ... FOR UPDATE), JSONB column support, and Point-In-Time Recovery (PITR).
In-Memory Cache & Key-Store	Redis	7.2	Ultra-fast in-memory key-value store for API rate-limiting, idempotency tokens (X-Idempotency-Key), and ephemeral session storage.

Architecture Tier	Primary Technology	Selected Version	Strategic Justification for Capstone & Production
Replenishment Algorithms	NumPy / SciPy	1.26+ / 1.13+	High-precision vector math for calculating Greasley's dual-variance safety stock and statistical Z-score quantiles.
Shrinkage Anomaly Detection	Scikit-learn	1.4+	Unsupervised IsolationForest model to detect anomalous stock discrepancy clusters across shifts and zones.
Reverse Proxy & Gateway	Nginx	1.26 (Alpine)	High-performance reverse proxy handling TLS 1.3 termination, gzip compression, static asset serving, and request rate-limiting.
Containerization	Docker & Compose	26.0+ / v2.27+	Guaranteed development-to-production parity; single-command local multi-container spin-up (docker compose up -d).
Concurrency Load Testing	Locust	2.28+	Distributed Python-based load testing simulating 50+ concurrent cashier POS deductions to prove sub-2.0s SLA compliance.

2. Architecture Decision Records (ADRs)

ADR-01: Relational PostgreSQL vs. NoSQL MongoDB

- **Status:** ACCEPTED
- **Context:** The system manages physical inventory, financial purchase orders, supplier credit notes, and store billing transactions.
- **Decision:** We select **PostgreSQL 16** as the core database engine instead of a NoSQL document database (like MongoDB).
- **Rationale:**
 1. *ACID Compliance & Ledger Integrity:* A warehouse and POS system requires strict ACID properties. Stock deduction and financial ledger updates cannot afford eventual consistency anomalies where units "disappear" or negative stock is accepted.
 2. *Relational Constraints:* Enforcing referential integrity (e.g., preventing deletion of an active SKU when inventory batches exist via [ON DELETE RESTRICT](#)) is handled natively by the database engine, preventing orphaned records.
 3. *Row-Level Locking:* PostgreSQL offers world-class row-level locking ([SELECT ... FOR UPDATE](#)) essential for resolving concurrent cashier checkouts.

- **Consequences:** Schema changes require structured migrations (via Alembic). Horizontal sharding is more complex than NoSQL, but acceptable given that a single normalized PostgreSQL instance with B-Tree indexes easily supports tens of millions of records.

ADR-02: Pessimistic Row-Level Locking vs. Optimistic Concurrency Control

- **Status:** ACCEPTED
- **Context:** High-velocity FMCG items (e.g., 1-Litre Soyabean Oil, Milk Vita cartons) experience multiple simultaneous checkout deductions across different retail registers.
- **Decision:** We implement **Pessimistic Row-Level Locking** (``SELECT ... FOR UPDATE``) for stock deductions, rather than Optimistic Concurrency Control (OCC with version column).
- **Rationale:**
 4. *Retry Storms in OCC:* Under high contention (such as a festival rush with 5 cashiers selling the same fast-moving item simultaneously), OCC causes 4 out of 5 transactions to fail and trigger application-layer retries. This creates compounding latency spikes and risks user frustration.
 5. *Deterministic Queueing:* Pessimistic row locking briefly serializes only the specific batch row for < 25 ms. Each cashier transaction waits a few milliseconds, decrements stock, and completes cleanly without rollback exceptions.
- **Consequences:** Requires disciplined lock ordering (sorting requested SKUs in ascending order) to completely prevent deadlock scenarios.

ADR-03: Progressive Web Application (PWA) vs. Native Android/Kotlin APK

- **Status:** ACCEPTED
- **Context:** Warehouse floor operators and receiving clerks need mobile barcode scanning interfaces on handheld terminals.
- **Decision:** We build the mobile operator interface as a **Progressive Web Application (PWA)** in Next.js/React, rather than a native Android Kotlin application.
- **Rationale:**
 6. *Frugal Hardware Agility:* Industrial Android terminals (Zebra, Honeywell) cost upwards of 60,000 BDT. A responsive PWA runs smoothly on standard consumer smartphones (Samsung, Xiaomi, Symphony) paired with 3,500 BDT Bluetooth barcode trigger handles.
 7. *Zero App-Store Friction & Instant Updates:* Warehouse floor workers do not need to install APKs or configure MDM systems. Any bug fix or algorithm update deployed to the web server is instantly available to all floor devices upon refresh.
 8. *Offline Resilience via Service Workers:* Modern Web APIs (Cache Storage API and IndexedDB) provide robust offline capabilities comparable to native SQLite storage.
- **Consequences:** Camera barcode scanning in low-light environments requires high-quality camera autofocus, which is mitigated by supporting physical Bluetooth HID scanner grips.

ADR-04: Python FastAPI vs. Node.js Express for Core Backend

- **Status:** ACCEPTED
- **Context:** The backend must handle high-concurrency I/O (barcode scans and POS sync) while simultaneously executing complex mathematical operations (Greasley's Safety Stock, EOQ, and Machine Learning anomaly detection).

- **Decision:** We select **Python 3.11+ with FastAPI** as the primary backend framework.
 - **Rationale:**
9. *Scientific & Algorithmic Ecosystem:* Implementing Greasley's dual-variance formula and Isolation Forest anomaly models in Node.js requires awkward third-party wrappers or spawning Python subprocesses. FastAPI natively executes NumPy, SciPy, and Scikit-learn within the same runtime.
 10. *Asynchronous High-Throughput I/O:* FastAPI running on **uvloop** matches or exceeds Node.js Express throughput for I/O-bound database queries.
 11. *Pydantic v2 Type Safety:* Automatic request validation, serialization, and interactive Swagger UI documentation out of the box.
- **Consequences:** Developers must write asynchronous code (**async/await**) properly to avoid blocking the event loop during heavy CPU calculations (heavy math is offloaded to Celery background workers).

ADR-05: Stateless JWT Tokens with Rotated Refresh Cookies

- **Status:** ACCEPTED
 - **Context:** The system must securely authenticate hundreds of mobile scanner sessions, desktop management consoles, and retail POS cash registers.
 - **Decision:** We implement a **Stateless Dual-Token Architecture**: Short-lived JWT Access Tokens (15 minutes) combined with HttpOnly, Secure, SameSite Refresh Tokens (8 hours) with automated refresh token rotation.
 - **Rationale:**
12. *Stateless API Gateway:* The API Gateway verifies token signatures locally without querying the database for every single barcode scan, guaranteeing sub-second latency.
 13. *Mitigation of XSS and CSRF:* Sensitive refresh tokens are stored in HttpOnly cookies unreachable by malicious client-side JavaScript, while access tokens reside in memory.
 14. *Immediate Revocation on Theft:* When a refresh token is used, it is rotated. If a compromised refresh token is reused, all tokens in that session family are immediately invalidated in Redis.
- **Consequences:** Clock synchronization (NTP) across servers is required to ensure consistent JWT expiry validation.

3. Infrastructure & Deployment Environment Specifications

3.1 Local Development Environment

- **OS:** Linux (Ubuntu 22.04+ / Debian 12) or macOS / Windows WSL2.
- **Runtimes:** Python **3.11.9+**, Node.js **18.20+** (LTS), Docker **26.0+**.
- **Database Tools:** Beekeeper Studio, pgAdmin 4, or DBeaver.
- **Hardware Requirements:** Minimum 8 GB RAM (16 GB recommended), 4-Core CPU, 20 GB free disk space.

3.2 Docker Compose Service Ports & Network Topology

```
version: '3.8'

services:
  retailsync-db:
    image: postgres:16-alpine
    container_name: retailsync-db
    environment:
```

```

POSTGRES_DB: retailsync_db
POSTGRES_USER: retailsync_user
POSTGRES_PASSWORD: retailsync_secure_password_2026
ports:
  - "5432:5432"
volumes:
  - pgdata:/var/lib/postgresql/data
  - ./docs/DATABASE_SCHEMA.sql:/docker-entrypoint-initdb.d/init.sql

retailsync-redis:
  image: redis:7.2-alpine
  container_name: retailsync-redis
  ports:
    - "6379:6379"
  command: ["redis-server", "--appendonly", "yes"]

retailsync-backend:
  build: ./backend
  container_name: retailsync-backend
  environment:
    DATABASE_URL: postgresql+asyncpg://retailsync_user:retailsync_secure_password_2026@retailsync-
db:5432/retailsync_db
    REDIS_URL: redis://retailsync-redis:6379/0
    JWT_SECRET: super_secure_dev_jwt_secret_key_diu_2026
  ports:
    - "8000:8000"
  depends_on:
    - retailsync-db
    - retailsync-redis

retailsync-frontend:
  build: ./frontend
  container_name: retailsync-frontend
  environment:
    NEXT_PUBLIC_API_URL: http://localhost:8000/api/v1
  ports:
    - "3000:3000"
  depends_on:
    - retailsync-backend

volumes:
  pgdata:

```